GenoKnoxDutch

Integration of GenoCAD and Knox and Double Dutch

Varsha Athreya, Melissa Regalado, Paulina Garcia, Denalda Gashi

Project Overview

Synthetic biology involves designing DNA constructs from reusable parts: promoters, ribosome binding sites, coding sequences, and terminators.

- GenoCAD represents these designs using formal grammar rules.
- Knox enumerates all valid combination within a defined design space.

The goal is to build a tool that extracts GenoCAD grammar from the SQL database, translate into goldbar expressions, and allow the user to explore valid designs through a browser interface.

Example: A researcher designing a gene therapy vector can select desired parts: (promoters, genes, and terminators) and then instantly generate all the valid DNA circuit combinations.

Functionality

1

Extract

Read a GenoCAD SQLite database and pull out every genetic part grouped by its biological role (promoter, RBS, CDS, terminator).

2

Translate

Convert the extracted parts into a Goldbar expression which is the language Knox uses to describe combinatorial design spaces.

3

Explore

Serve a browser UI where a researcher checks off which parts to include, sees the Goldbar update live, and hits Generate to get every valid design from Knox.

4

Export

Download results as a CSV: selected parts list or the full enumerated design table for use in downstream wet-lab planning tools.

Design Specification

Requirements:

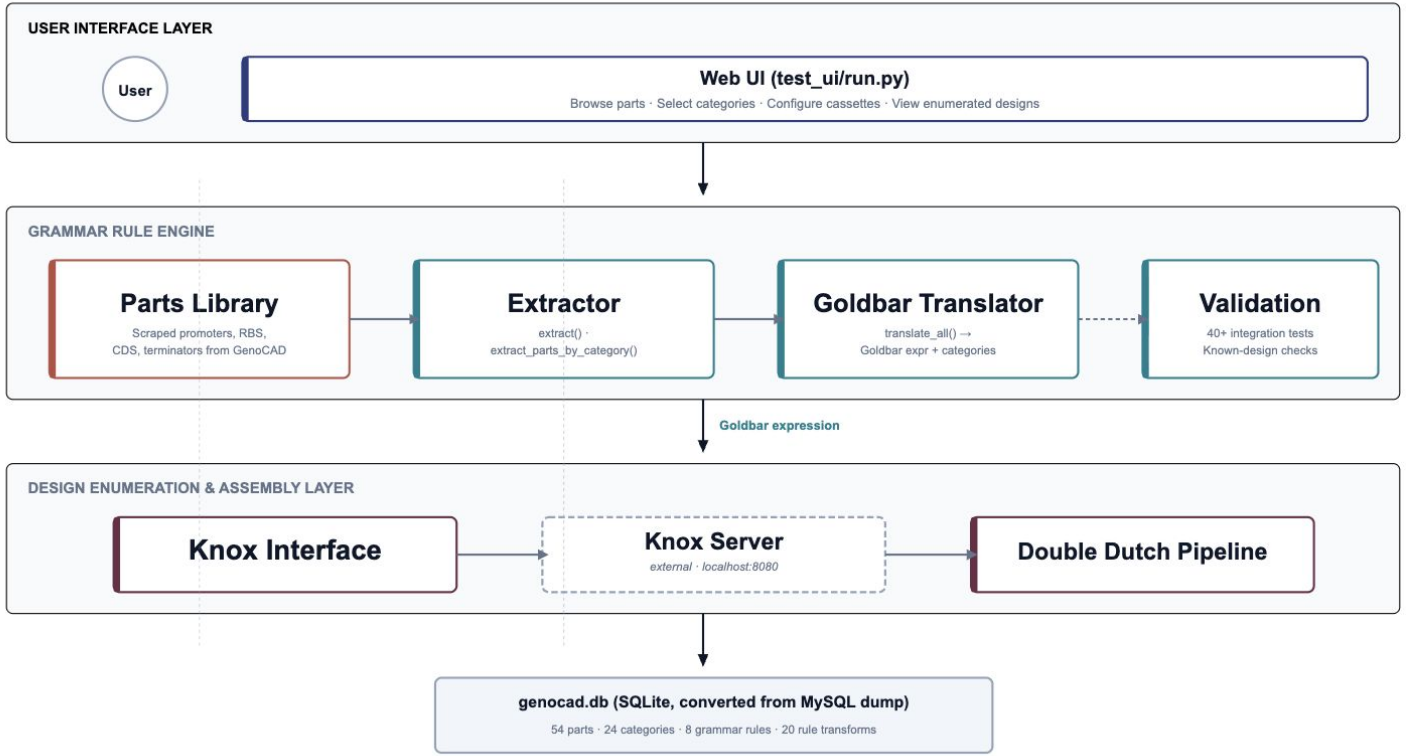
- Read genetic parts from the GenoCAD SQL database.
- Generate valid Goldbar expressions automatically.
- Communicate with Knox API.
- Run fully locally.

Constraints:

- Must proxy Knox requests.
  - Browsers block CORS so the server must act as a same-origin proxy.
- Goldbar grammar must match Knox’s expected format.
  - Knox expects specific SBOL role names.
- Support 1 or multi-cassette designs

GenoCAD → Knox → Double Dutch Pipeline

Block-level architecture of our synthetic biology design tool



GenoCAD Database

- In the parts table, each row is one genetic part with an ID, description, and DNA sequence.
- Read three tables (parts, categories, categories_parts), join them together and then produce a Python dictionary {category_name: [list of parts]}.
- Skip some categories that are structural grammar tokens such as S (grammar start symbol).
- Output a table with the ID and description per part.

Table: parts

	id	part_id	description	segment	user_id
	Filter	Filter	Filter	Filter	Filter
12	16034	a0251	R0011	aattgtgagcggataacaattgacattgtgagcgga…	0
13	16035	a025m	R0085 - T7 Consensus Promoter	taatacgactcactatagggaga	0
14	16036	a025n	Ptrc-2	ccatcgaatggctgaaatgagctgttgacaattaat…	0
15	16037	a025o	Pl-s1con	gcatgcacagataaccatctgcggtgataaattatc…	0
16	16038	a025p	Pltet0-1	gcatgctccctatcagtgatagagattgacatccct…	0
17	16039	a025q	RBS A	aggaggaaaaaa	0
18	16040	a025r	RBS B	aggaatttaa	0
19	16041	a025s	RBS C	aggaaacagacc	0
20	16042	a025t	RBS D	aggaaaccggttcg	0
21	16043	a025u	RBS E	aggaaaccggtt	0
22	16044	a025v	RBS F	aggacggttcg	0
23	16045	a025w	RBS G	aggaaaggcctcg	0
24	16046	a025x	RBS H	aggacggccgg	0
25	16047	a025y	E1010 - RFP Mutant	atggcttcctccgaagacgttatcaaagagttcatg…	0
26	16048	a025z	J45004 - salicylic acid carboxyl …	atggaagttgttgaagttcttcacatgaatggagga…	0
27	16049	a0260	E0020 - eCFP	atggtgagcaagggcgaggagctgttcaccggggtg…	0
28	16050	a0261	E0030 - eYFP	atggtgagcaagggcgaggagctgttcaccggggtg…	0
29	16051	a0262	J36801 - KaiA coding region	gtgctctcgcaaattgcaatctgcatttgggtggaa…	0
30	16052	a0263	J36804 - KaiC coding region	atgacttccgctgagatgactagccctaataataat…	0
31	16053	a0264	J31007 - tetracycline resistance …	atgaaatctaacaatgcgctcatcgtcatcctcggc…	0
32	16054	a0265	J52008 - luciferase	atggcttccaaggtgtacgaccccgagcaacgcaaa…	0
33	16055	a0266	E0040 - GFP Wild Type	atgcgtaaaggagaagaacttttcactggagttgtc…	0
34	16056	a0267	J06501 - LacI repressor ts	atggtgaatgtgaaaccagtaacgttatacgatgtc…	0
35	16057	a0268	cIts	atgagcacaaaaaagaaaccattaacacaagagcag…	0
36	16058	a0269	TetR	atgtctagattagataaaagtaaagtgattaacagc…	0
37	16059	a026a	tgGFPmut3	atgggtaccatgagtaaaggagaagaactttcact…	0
38	16060	a026b	B0010 - T1 from E. coli rrnB	ccaggcatcaaataaaacgaaaggctcagtcgaaag…	0
39	16061	a026c	B0012	tcacactggctcaccttcgggtgggcctttctgcgt…	0
40	16062	a026d	B0016 - T7 terminator	ctagcataaccccttggggcctctaaacgggtcttg…	0

Goldbar Translator

Categories Payload

- Describes which parts fill each category.

```
{
  "Promoter": {
    "promoter":
      ["a069g", "a069h"]
  },
  "Ribosome": {
    "ribosomeBindingSite":
      ["r001"]
  },
  "Coding_sequence": {
    "cds": ["c001","c002"]
  }
}
```

Goldbar Expression

- Describes how genetic parts can be assembled.

```
Promoter . Ribosome . Coding_sequence . Terminator
```

The . dot means "followed by" since order is fixed by biology.

Multi-cassette variant:

```
one-or-more(
  Promoter . Ribosome . Coding_sequence . Terminator)
```

Tells Knox to repeat the cassette 1, 2, 3+ times.

Knox needs the structure and the content separately so it can apply the grammar rules before looking at specific parts. Goldbar = the recipe template. Categories = the actual parts.

Goldbar Translator

```
selected = {
  "Promoter":
    ["a069g","a069h"],
  "Ribosome":
    ["r001"],
  "Coding sequence":
    ["c001","c002"],
  "Terminator":
    ["t001"]
}
```

→

① Order

Sort into biological order:

```
Promoter
Ribosome
Coding_seq
Terminator
```

→

② Sanitize

Spaces → underscores:

```
"Coding sequence"
→ Coding_sequence
```

Knox grammar breaks on spaces in token names

→

③ SBOL Role

Wrap in standard role:

```
"Promoter": {
  "promoter":
    ["a069g"...]
}
```

"promoter" = SBOL international standard

Result: `Promoter . Ribosome . Coding_sequence . Terminator` + categories JSON → Knox enumerates valid designs

Knox combines both to enumerate every valid part combination.

GenoCad –> Goldbar Verification

Script	Purpose
convert_to_db.py	Rebuilds SQLite DB from the legacy MySQL dump
genocad_grammar_extractor.py	Reads the SQLite DB and pulls out two things: **(1) grammar rules** via **extract(), (2) parts** grouped by category via **extract_parts_by_category()**.
goldbar_translator.py	Takes the extracted parts dict and converts it to a Goldbar expression string plus Knox-compatible categories dict **via translate_all().**
test_db_conversion.py	Verifies row counts match the source
test_sequence_integrity.py	Catches DNA sequence corruption
test_known_designs.py	Validates extractor against published circuits
test_goldbar_validation.py	Validates translator output format
test_integration.py	End-to-end pipeline check

GenoCad -> Goldbar Verification

```
============================================================
 GENOCAD -> GOLDBAR INTEGRATION TEST
============================================================

============================================================
 1. DATABASE CONVERSION: row counts
============================================================
  [PASS] database file exists - /Users/melissaregalado/Desktop/EC552_Project/
  [PASS] categories row count - 24 rows (expected 24)
  [PASS] parts row count - 54 rows (expected 54)
  [PASS] categories_parts row count - 54 rows (expected 54)
  [PASS] libraries row count - 2 rows (expected 2)
  [PASS] rules row count - 8 rows (expected 8)
  [PASS] rule_transform row count - 20 rows (expected 20)
  [PASS] grammars row count - 2 rows (expected 2)

============================================================
 2. SCHEMA: required tables present
============================================================
  [PASS] table 'categories' exists
  [PASS] table 'parts' exists
  [PASS] table 'categories_parts' exists
  [PASS] table 'libraries' exists
  [PASS] table 'rules' exists
  [PASS] table 'rule_transform' exists
  [PASS] table 'grammars' exists
  [PASS] table 'library_part_join' exists
  [PASS] table 'designs' exists

============================================================
 3. DNA SEQUENCE INTEGRITY
============================================================
  [PASS] a069g (placI) sequence length - 200 bp (expected 200)
  [PASS] a069h (pcI) sequence length - 49 bp (expected 49)
  [PASS] a069i (pTetR) sequence length - 54 bp (expected 54)
  [PASS] a069m (My_RBS) sequence length - 12 bp (expected 12)
  [PASS] a069n (B0010) sequence length - 80 bp (expected 80)

============================================================
```

Row Counts & catches 7 key tables for SQL conversion

Checks all required tables exist in .db

5 sequences have exact expected bp lengths & Catches character-level corruption in genetic data

GenoCad -> Goldbar Verification

```
============================================================
 4. GRAMMAR RULE EXTRACTION
============================================================
  [PASS] rules returned for grammar 1237 - 4 rules
  [PASS] rule '1cas' present
  [PASS] rule '2cas' present
  [PASS] rule '3cas' present
  [PASS] rule 'Transcription' present
  [PASS] Transcription rule sequence = [PRO, RBS, CDS, TER] - got ['PRO', 'RBS', 'CDS', 'TER']

============================================================
 5. PARTS EXTRACTION BY CATEGORY
============================================================
  [PASS] PRO category present - 3 parts
  [PASS] RBS category present - 1 parts
  [PASS] CDS category present - 3 parts
  [PASS] TER category present - 1 parts
  [PASS] part dict has 'id' field
  [PASS] part dict has 'desc' field
  [PASS] part dict has 'seq' field

============================================================
 6. GOLDBAR TRANSLATION: single cassette
============================================================
  [PASS] single-cassette expression is 'PRO . RBS . CDS . TER' - got 'PRO . RBS . CDS . TER'
  [PASS] PRO mapped to Knox role 'promoter'
  [PASS] RBS mapped to Knox role 'ribosomeBindingSite'
  [PASS] CDS mapped to Knox role 'cds'
  [PASS] TER mapped to Knox role 'terminator'

============================================================
 7. GOLDBAR TRANSLATION: multi-cassette
============================================================
  [PASS] multi-cassette wraps in 'one-or-more(...)' - got 'one-or-more(PRO . RBS . CDS . TER)'
```

Rules parse correctly, Transcription = [PRO, RBS, CDS, TER]. Confirms biological ordering is preserved

Each category (PRO/RBS/CDS/TER) returns non-empty parts.Ensures Knox gets usable input

Output equals "PRO . RBS . CDS . TER"Verifies basic translation syntax

Wraps correctly as "one-or-more(...)"Verifies Knox-compatible repeatable constructs

GenoCad -> Goldbar Verification

```
============================================================
 8. KNOWN DESIGN: Switch LacI/TetR (design_id 673)
============================================================
  [PASS] PRO.promoter contains a069g
  [PASS] RBS.ribosomeBindingSite contains a069m
  [PASS] CDS.cds contains a069l
  [PASS] TER.terminator contains a069n

============================================================
 9. KNOWN DESIGN: Repressilator (design_id 674)
============================================================
  [PASS] PRO.promoter contains a069g
  [PASS] PRO.promoter contains a069h
  [PASS] PRO.promoter contains a069i
  [PASS] RBS.ribosomeBindingSite contains a069m
  [PASS] CDS.cds contains a069j
  [PASS] CDS.cds contains a069k
  [PASS] CDS.cds contains a069l
  [PASS] TER.terminator contains a069n

============================================================
 FINAL SUMMARY
============================================================

  Total checks: 53
  Passed:       53
  Failed:       0

  ALL TESTS PASSED
============================================================
```

Known design: Switch LacI/TetR. All 4 parts of *Gardner, 2000* circuit retrievable.
What it does: biological version of a flip-flop circuit.
Why it matters: cells can now "remember" a signal after the trigger is gone.

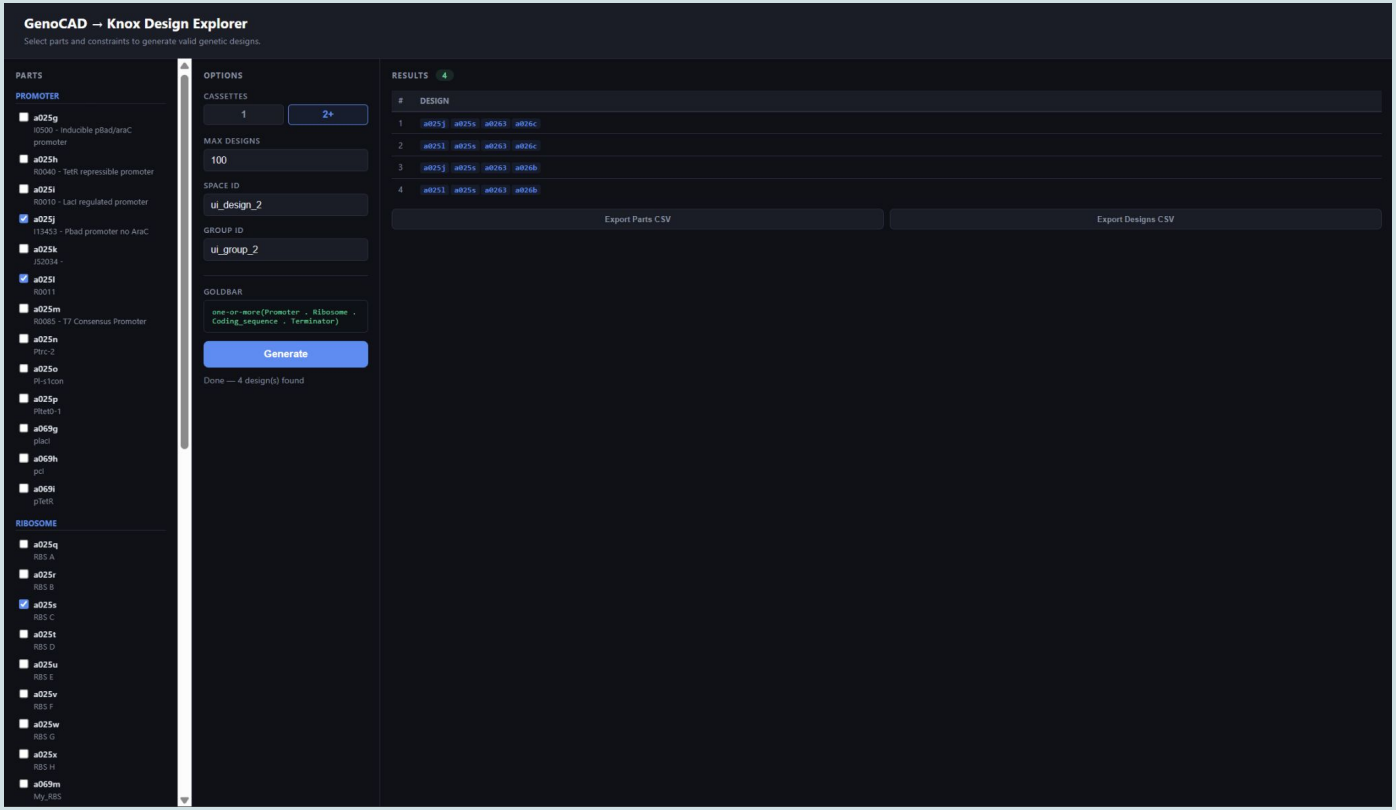
All 8 parts of *Elowitz & Leibler, 2000* circuit retrievable.
Why it matters: first synthetic oscillator made of genes.
What it does: The cell produces green fluorescent protein (GFP) that pulses on and off rhythmically, like a clock.

- 7 row-count checks
- 9 schema existence checks
- 5 DNA sequence length checks
- 5 grammar rule checks
- 7 parts extraction checks
- 5 single-cassette Goldbar checks
- 1 multi-cassette Goldbar check
- 4 Switch LacI/TetR checks
- 10 Repressilator checks

Knox Connection

- Knox is a separate programs that runs locally on port 8080
  - Our application communicates with Knox using HTTP requests via REST API.
- POST /goldbar/import : sends the goldbar expression + categories payload so that Knox can build a design space graph internally and store it under the space and group id the user selects.
- GET /designSpace/enumerate : Knox returns every valid design as a JSON array.
- Knox API doesn’t include CORS configuration so this blocks direct requests from the browser.
- Built a Python HTTP server as a middle layer so the browser sends requests to our server and then the server forwards requests to Knox and returns results to server which sends the results back to the browser

Success - One Example



Success

- Built a complete end-to-end pipeline: GenoCAD parts -> Goldbar -> CSV of valid designs.
- Integrated multiple systems (SQLite, Python, HTML/JS).
- Successfully generated valid genetic designs in real time.
- Developed an interactive UI to let users choose parts and see valid designs.
- Solved the CORS problem without modifying Knox.
- The extractor correctly filters out structural grammar tokens so the user can only see real biological parts.

Failures

- Still experimenting to see if DoubleDutch can be integrated:
  - DoubleDutch requires part strength (RPU values)/quantitative biological data which the GenoCAD database did not have.
  - This created a mismatch between the design generation and design optimization
  - Possible solution would be to scrape through other sources such as iGEM registry and extract parts with assigned strength/RPU values.
- Add more validation checks needed since we are working with a different application, we want to make sure that if there is an error with Knox that the error can be communicated to the user.
- Add more parts (iGEM) and grammar rules (from research)